EECS 484, Winter 2026 Database Management Systems

SQL

Chapter 5

Instructor: Barzan Mozafari

University of Michigan, Ann Arbor

Structured Query Language

- Create/Drop tables `CREATE/DROP TABLE(DDL)`
- Create/Update view `CREATE/DEOP VIEW (DDL)`
- Add/Del records `INSERT/DELETE (DML)`
- Update records `UPDAET (DML)`
- Retrieve records `SELECT (DQL)`
- Other controls `COMMIT/ROLLBACK (DCL in Oracle)`

SQL Query Language

- Implements relational algebra…

  Select, Project, Join, Set operators

"Relationally complete"

- And much more…
  - Correlated subqueries
  - Ordering of results
  - Aggregate queries (e.g., SUM, MAX, AVG)
  - Three-valued logic for NULL values
  - Etc.

Basic SQL Query

Optional

Attributes from input relations

```
SELECT [DISTINCT] attr-list
FROM relation-list
WHERE qualification
```

List of relations

Attr1 op Attr2
OPS: <, >, =, <=, >=, <>
Combine using AND, OR, NOT

(Conceptual) Evaluation:

1. Take cross-product of relation-list
2. Select rows satisfying qualification
3. Project columns in attr-list
   (eliminate duplicates only if DISTINCT)

Optimizer chooses efficient plan!

Example

- Schema:
  - Sailors (sid, sname, rating, age)
  - Boats (bid, bname, color)
  - Reserves (sid, bid, rday)
- Query: Find the names of sailors who have reserved boat #103

```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid = R.sid AND R.bid = 103;
```

- R and S are called Range Variables and good style to use

Example of Basic Query:

CROSS-PRODUCT SYNTAX:

```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid = R.sid AND
R.bid = 103;
```

Reserves

sid	bid	rday
22	101	10/10
58	103	11/12

Sailors

sid	sname	rating	age
22	Dustin	7	45
58	Rusty	10	35
31	Lubber	8	55

Reserves x Sailors

sid	bid	rday	sid	sname	rating	age
22	101	10/10	22	Dustin	7	45
22	101	10/10	58	Rusty	10	35
22	101	10/10	31	Lubber	8	55
58	103	11/12	22	Dustin	7	45
58	103	11/12	58	Rusty	10	35
58	103	11/12	31	Lubber	8	55

Range Variables

Query: Find the names of sailors who have reserved boat #103

- R and S are Range Variables and used to name relations in the query. Good style to use them.

With Range Variables R and S	Without Range Variables
SELECT S.sname FROM Sailors S, Reserves R WHERE S.sid = R.sid AND R.bid = 103;	SELECT sname FROM Sailors, Reserves WHERE Sailors.sid = Reserves.sid AND Reserves.bid = 103;

Quiz

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Dustin	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid = R.sid AND
R.bid = 103;
```

The result of executing this query will be:

A. {Dustin, Lubber}
B. {Dustin, Lubber, Dustin}

Answer

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Dustin	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

{Dustin,
Lubber,
Dustin}

```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid = R.sid AND
R.bid = 103;
```

Eliminating Duplicates

```
SELECT DISTINCT sname, bid
FROM Sailors S, Reserves R
WHERE S.sid = R.sid;
```

John, 2
~~John, 2~~
John, 3
Mary, 2

- Add keyword DISTINCT after SELECT to eliminate duplicates from the result
- By default, SQL allows duplicate rows in the result (multi-set semantics), unlike ER diagrams and Relational Algebra, for efficiency.

Note on Range Variables

Always needed when same relation appears twice in FROM clause

```
SELECT S1.sname, S2.sname
FROM Sailors S1, Sailors S2
WHERE S1.age > S2.age;
```

What does this Query compute?

It is considered good style to use range variables even when not needed.

INNER Join Syntax

The query we just saw is really computing a join or an inner join between relations R and S on the common attribute sid.

(We will see outer joins shortly)

Join syntax:

```
SELECT S.sname
FROM Sailors S JOIN Reserves R ON S.sid = R.sid
WHERE R.bid = 103;
```

I tried to tell a joke about JOINs…
…but not everyone could relate.

Eqvt. Inner join syntax:

```
SELECT S.sname
FROM Sailors S INNER JOIN Reserves R ON
     S.sid = R.sid
WHERE R.bid = 103;
```

INSERT INTO using SELECT

- Goal: we want to populate a table from existing data that may need some cleanup or sanitzation.
- Example: Create a table MySailors from the Sailors table, but do not include age and eliminate any sailors whose name is missing.
  - We assume MySailors(<u>sid</u>, sname, rating) has been defined.

```
INSERT INTO MySailors
SELECT S.sid, S.sname, S.rating
FROM Sailors S
WHERE S.sname IS NOT NULL;
```

Find the colors of boats reserved by sailors named Rusty

- Schema:

```
Sailors (sid, sname, rating, age)
Boats (bid, bname, color)
Reserves (sid, bid, rday)
```

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

```
SELECT B.color
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND
      S.sname = 'Rusty';
```

Find pairs of sailors where the first one has half the rating of the second one

```
SELECT S1.sname AS name1, S2.sname AS name2
FROM Sailors S1, Sailors S2
WHERE 2*S1.rating = S2.rating;
```

(Notice the column renamings)

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Arithmetic Operations

- Compute ratings+1 of persons who have sailed two different boats on the same day:

```
SELECT S.sname, S.rating+1 AS rating
FROM Sailors S, Reserves R1, Reserves R2
WHERE S.sid=R1.sid AND S.sid=R2.sid AND
      R1.day=R2.day AND R1.bid<>R2.bid;
```

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

ORDER BY Clause

Sorts the result for presentation

Attribute(s) in ORDER BY clause (must be) in SELECT list

Find the names
and ages of all sailors,
in increasing order of age

```
SELECT S.sname, S.age
FROM Sailors S
ORDER BY S.age [ASC]
```

Find the names
and ages of all sailors,
in decreasing order of age

```
SELECT S.sname, S.age
FROM Sailors S
ORDER BY S.age DESC
```

ORDER BY Clause

```
SELECT S.sname, S.age, S.rating
FROM Sailors S
ORDER BY S.age ASC, S.rating DESC
```

What does this query compute?

Find the names, ages, and rankings of all sailors.

Sort the result in <u>increasing</u> order of age.

If there is a tie, sort those tuples in decreasing order of rating.

What We Will Learn Next

- Set operators
- Set comparison
- Nested queries
- Aggregation

Set Operators

- UNION (eliminates duplicates)
- INTERSECT (eliminates duplicates)
- EXCEPT or MINUS (set difference, eliminates duplicates)

- UNION ALL (keeps duplicates)

less common

- INTERSECT ALL:
- EXCEPT ALL:

Duplicate-Preserving Set Operators

- *Table A: (1, 1, 1, 2)*
- *Table B: (1, 1, 3)*

- A UNION ALL B → (1, 1, 1, 2, 1, 1, 3) *-- sum of counts*
- A INTERSECT ALL B → (1, 1) *-- min of counts*
- A EXCEPT ALL B → (1, 2) *-- A's count minus B's count*

Union Example

Find names of sailors who have reserved a red or a green boat.

Try without UNION

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Union Example

Find names of sailors who have reserved a red or a green boat.

```
SELECT  DISTINCT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid
  AND (B.color = 'red' OR B.color = 'green');
```

```
SELECT  S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
UNION
SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid and R.bid = B.bid AND B.color = 'green';
```

Intersection Question

- Find names of sailors who have reserved a red boat and a green boat.

```
SELECT  S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid
  AND (B.color = 'red' AND B.color = 'green');
```

What is wrong with the above query?

A. Extra parentheses on the last line should not be there
B. A boat cannot be multi-colored.
   A boat with red and green stripes would not satisfy last line
C. Both A and B above
D. Neither. There is nothing wrong.

Answer

- Find names of sailors who have reserved a red boat and a green boat.

```
SELECT  S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid
  AND (B.color = 'red' AND B.color = 'green');
```

The integrity constraint that bid is a key tells us that the same boat cannot have two colors…

A. Extra parentheses on the last line should not be there
B. A boat cannot be multi-colored.
   A boat with red and green stripes would not satisfy last line
C. Both A and B above
D. Neither. There is nothing wrong.

Intersect – More solutions

Find names of sailors who have reserved a red and a green boat.

```
SELECT  S.sname
FROM Sailors S, Reserves R1, Boats B1,
     Reserves R2, Boats B2
WHERE S.sid = R1.sid AND R1.bid = B1.bid
  AND S.sid = R2.sid AND R2.bid = B2.bid
  AND B1.color = 'red' AND B2.color = 'green’
```

```
SELECT  S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
INTERSECT
SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid and R.bid = B.bid AND B.color = 'green';
```

Bug?

Intersect – Buggy solution

Find names of sailors who have reserved a red and a green boat.

```
SELECT  S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red’

INTERSECT

SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid and R.bid = B.bid AND B.color = 'green';
```

Bug?

Problem: Intersect on Non-Key

Find the names of sailors who have reserved a red and a green boat

```
SELECT  S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
INTERSECT
SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid and R.bid = B.bid AND B.color ='green'
```

Fix

1) intersecting on Sailor's sid (a key)
(2) Then map the sid values to names

Error Fixed – Intersect on key

Find the names of sailors who have reserved a red and a green boat

```
CREATE VIEW RedGreenSailors AS
SELECT  S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
INTERSECT
SELECT S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid and R.bid = B.bid AND B.color = 'green';

SELECT S.sname
FROM Sailors S, RedGreenSailors R
WHERE S.sid = R.sid;
DROP VIEW RedGreenSailors;
```

VIEWS

Views are used for creating external schemas or temporary schemas on top of conceptual schema.

```
CREATE VIEW RedGreenSailors AS
SELECT  S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
INTERSECT
SELECT S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid and R.bid = B.bid AND B.color = 'green';
```

Further queries can use views like any other relation

A Solution without View

Substitute the view reference with the query and give it a name inline.

```
SELECT S.sname
FROM Sailors S,
(SELECT  S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
INTERSECT
SELECT S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid and R.bid = B.bid AND B.color = 'green')
RedGreenSailors
WHERE S.sid = RedGreenSailors.sid;
```

Set Difference Example

Find tuples in A that are not in B

```
SELECT * FROM A
MINUS
SELECT * FROM B;
```

MINUS and EXCEPT are synonyms

Set Difference Example

Find sids of sailors who have reserved red, but not green boats.

```
SELECT  S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid
      AND B.color = 'red'
EXCEPT
SELECT  S.sid
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid
      AND B.color = 'green'
```

Simpler?

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Set Difference Example

Find sids of sailors who have reserved red, but not green boats.

```
SELECT  R.sid
FROM Reserves R, Boats B
WHERE R.bid = B.bid AND B.color = 'red'
EXCEPT
SELECT  R.sid
FROM Reserves R, Boats B
WHERE R.bid = B.bid AND B.color = 'green'
```

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

More Set Comparison Operators

- Set comparisons:
  - *(attr1, …, attrN)* IN R : true if R contains *tuple (attr1, …, attrN)*
  - EXISTS R : true if R is not an empty relation
  - UNIQUE R : true if no duplicates in R
  - You can use NOT with these, e.g. NOT EXISTS

- Also available ANY or ALL: (op is <, ≤, >, ≥, =, ≠)
  - *attr* > ANY R : some element of R satisfies the condition that attr > that element
  - *attr* < ALL R : all elements of R satisfy the condition that attr < element

Nested Queries

Query with another query embedded inside

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid IN
       (SELECT  R.sid
        FROM  Reserves R
        WHERE  R.bid=103)
```

What does this query compute?

Conceptual evaluation:

For each row of Sailors, evaluate the subquery over reserves.

To find sailors who have not reserved 103, use NOT IN.

Over-Use of Nesting

- Common error by novice SQL programmers
- Query optimizers not as good at optimizing queries across nesting boundaries
- Try hard first to write non-nested using joins

```
SELECT  DISTINCT S.sname
FROM Sailors S, Reserves R
WHERE S.sid = R.sid AND R.bid = 103;
```

Example

- Q1: What does this query compute?

```
SELECT S.sid
FROM Sailors S
WHERE S.rating > ANY (SELECT S2.rating
                      FROM Sailors S2
                      WHERE S2.name = 'John');
```

- Q2: Rewrite the query without using a nested query

Example

- Q1: What does this query compute?

```
SELECT S.sid
FROM Sailors S
WHERE S.rating > ANY (SELECT S2.rating
                      FROM Sailors S2
                      WHERE S2.name = 'John');
```

- Q2: Rewrite the query without using a nested query

```
SELECT DISTINCT S.sid
FROM Sailors S, Sailors S2
WHERE S.rating > S2.rating AND S2.name = 'John';
```

Example

Find sailors (all their info) whose rating is greater than that of all sailors named Horatio:

```
SELECT  *
FROM  Sailors S
WHERE  S.rating > ALL  (SELECT  S2.rating
                        FROM  Sailors S2
                        WHERE S2.sname='Horatio')
```

Question

```
SELECT  *
FROM  Sailors S
WHERE  S.rating > ALL  (SELECT  S2.rating
                        FROM  Sailors S2
                        WHERE S2.sname='Horatio')
```

What if there is no sailor with sname Horatio? Then, The "> ALL" condition is

A. True for every sailor in S. Returns all sailors.
B. False for every sailor in S. Returns empty relation.
C. Undefined

Answer

```
SELECT  *
FROM  Sailors S
WHERE  S.rating > ALL  (SELECT  S2.rating
                        FROM  Sailors S2
                        WHERE S2.sname='Horatio')
```

What if there is no sailor with sname Horatio? Then, The "> ALL" condition is

A. **True for every sailor in S. Returns all sailors.**
B. False for every sailor in S. Returns empty relation.
C. Undefined

Aggregate Operators

```
SELECT COUNT (*)
FROM Sailors S
```

```
SELECT COUNT
   (DISTINCT S.name)
FROM Sailors S
```

```
COUNT (*)
COUNT ( [DISTINCT] A)
SUM ( [DISTINCT] A)
AVG ( [DISTINCT] A)
MAX (A) Can use Distinct
MIN (A) Can use Distinct
```

single column*

```
SELECT AVG(S.age)
FROM Sailors S
WHERE S.rating=10
```

```
SELECT AVG(DISTINCT S.age)
FROM Sailors S
WHERE S.rating=10
```

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.rating = (SELECT  MAX(S2.rating)
                   FROM  Sailors S2)
```

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Aggregate Query - Example

Find name and age of the oldest sailor(s)

```
SELECT  S.sname, MAX (S.age)
FROM  Sailors S
```

How many tuples in the result?

```
SELECT  S.sname, S.age
FROM  Sailors S
WHERE  S.age =(SELECT  MAX (S2.age)
               FROM  Sailors S2)
```

Note sqlite3 doesn't support >= ALL syntax

```
SELECT  S.sname, S.age
FROM  Sailors S
WHERE  S.age >= ALL (SELECT  S2.age
                     FROM  Sailors S2)
```

GROUP BY

Conceptual evaluation

- Partition data into groups according to some criterion
- Evaluate the aggregate for each group

Example:
For each rating level, find the age of the youngest sailor

```
SELECT  S.rating, MIN (S.age)
FROM  Sailors S
GROUP BY  S.rating
```

How many tuples in the result?

GROUP BY and HAVING

```
SELECT [DISTINCT] target-list
FROM relation-list
WHERE qualification
GROUP BY grouping-list
HAVING group-qualification
ORDER BY column-list
```

Target-list contains:
1. Attribute names
(subset of grouping-list)
2. Aggregate operations
e.g. min(age)

Conceptual Evaluation:

1. **FROM:** Cartesian product of all rows from all tables
2. **WHERE:** Eliminate tuples that don't satisfy qualification
3. **GROUP BY:** Partition remaining rows into groups, calc. aggrs
4. **HAVING:** Eliminate groups according to group-qualification
5. **SELECT:** Evaluate aggregate operation(s) for each group
6. **ORDER BY:** order the output tuples

Quiz: Find the age of the youngest sailor with age >= 18, for each rating with at least 2 such sailors

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
~~71~~	~~Zorba~~	~~10~~	~~16.0~~
64	Horatio	7	35.0
29	Brutus	1	33.0
58	Rusty	10	35.0

Answer

```
SELECT  S.rating, MIN (S.age)
FROM  Sailors S
WHERE  S.age >= 18
GROUP BY  S.rating
HAVING  COUNT (*) >= 2
```

sid	sname	rating	age
29	Brutus	1	33.0
22	Dustin	7	45.0
64	Horatio	7	35.0
31	Lubber	8	55.5
58	Rusty	10	35.0

Grouped Sailors

ratin g	age
7	35.0

Answer relation

NULL Values in SQL

- NULL represents ‘unknown’ or ‘inapplicable’
- Query evaluation complications
  - Q: Is (rating > 10) true when rating is NULL?
  - A: Condition evaluates to ‘unknown’ (neither T nor F)
- What about AND, OR connectives?
  - Need 3-valued logic
- WHERE clause eliminates rows that don’t evaluate to true

p	q	p AND q	p OR q
T	T	T	T
T	F	F	T
T	U	U	T
F	T	F	T
F	F	F	F
F	U	F	U
U	T	U	T
U	F	F	U
U	U	U	U

Q: *Why do SQL developers hate nature?*
A: *Too many NULLs.*

More On NULL Values

- SUM, AVG, MIN, MAX, COUNT(columnName) all ignore NULL values
- COUNT(*) includes NULL values (does not ignore them)
- GROUP BY put all NULLs in the same group
  - Example GROUP by name, rating will put (NULL, NULL) in one group, (NULL, 3) in one group, (NULL, 4) in one group, (“Alex”, NULL) in one group and so on.

Quiz 1

Sailors

sid	sname	rating	age
22	Dustin	7	45
58	Rusty	10	NULL
31	Lubber	8	55

```
SELECT sname
FROM sailors
WHERE age > 45 OR age <= 45
```

What does this query return?

A. {Dustin, Rusty, Lubber}
B. {Dustin, Lubber}
C. {Lubber}
D. Error

Answer

Sailors

sid	sname	rating	age
22	Dustin	7	45
58	Rusty	10	NULL
31	Lubber	8	55

```
SELECT sname
FROM sailors
WHERE age > 45 OR age <= 45
```

What does this query return?

A. {Dustin, Rusty, Lubber}
B. {Dustin, Lubber}
C. {Lubber}
D. Error

Quiz 2

Sailors

sid	sname	rating	age
22	Dustin	7	45
58	Rusty	10	NULL
31	Lubber	8	55

```
SELECT AVG(age)
FROM sailors
```

What does this query return?

A. 50
B. NULL
C. Error
D. A range of possible values based on domain constraint on age

Answer

Sailors

sid	sname	rating	age
22	Dustin	7	45
58	Rusty	10	NULL
31	Lubber	8	55

```
SELECT AVG(age)
FROM sailors
```

What does this query return?

A. **50**
B. NULL
C. Error
D. A range of possible values based on domain constraint on age

Note: ALL aggregate operations (including COUNT except COUNT(*)) discard null values (so the answer is 50)

Outer Joins

Sailors

sid	sname	rating	age
22	dustin	7	45.0
58	rusty	10	35.0

Reserves

sid	bid	day
22	101	10/10/99

```
SELECT S.sid, R.bid
FROM Sailors S NATURAL LEFT[OUTER]
     JOIN Reserves R
```

Result

sid	bid
22	101
58	null

Similarly:

- Right Outer Join
- Full Outer Join

Note: OUTER is default, when using LEFT, RIGHT, or FULL

More Outer Joins

```
SELECT S.sid, R.bid
FROM Sailors S RIGHT[OUTER] JOIN
   Reserves R ON S.sid=R.sid;
```

Sailors

sid	sname	rating	age
58	Rusty	10	35
31	Lubber	8	55

Reserves

sid	bid	rday
22	101	10/10
58	103	11/12

Result

sid	bid
null	101
58	103

JOIN Syntax with Multiple Tables

Sailors

sid	sname	rating	age
22	dustin	7	45.0
58	rusty	10	35.0

Reserves

sid	bid	day
22	101	10/10/99

```
SELECT S.sname, B.bname
FROM Sailors S JOIN Reserves R ON (S.sid = R.sid)
               JOIN Boats B ON (R.bid = B.bid)
WHERE S.name = 'dustin';
```

Similarly:

- RIGHT [OUTER] JOIN ON…
- FULL [OUTER] JOIN ON…
- LEFT [OUTER] JOIN on…
- NATURAL JOINS (outer and inner)

For each red boat, find the number of reservations for this boat

```
SELECT B.bid,  COUNT (*) AS scount
FROM   Boats B, Reserves R
WHERE R.bid=B.bid AND B.color='red'
GROUP BY  B.bid
```

Is this query correct?

For each red boat, find the number of reservations for this boat

```
SELECT B.bid,  COUNT (R.bid) AS scount
FROM   Boats B LEFT JOIN Reserves R ON R.bid=B.bid
WHERE B.color='red'
GROUP BY  B.bid
```

```
SELECT B.bid,  COUNT (R.bid) AS scount
FROM   Boats B LEFT JOIN Reserves R ON R.bid=B.bid
GROUP BY  B.bid
HAVING B.color='red'
```

Would this work?
Note: one color per bid

Subtle Errors

Find the sid of sailors who have reserved exactly one boat

```
SELECT S1.sid FROM Sailors S1
MINUS
SELECT R1.sid
FROM Reserves R1, Boats B1, Reserves R2, Boats B2
WHERE R1.sid=R2.sid AND R1.bid=B1.bid
      AND R2.bid=B2.bid AND R1.bid <> R2.bid;
```

There is a subtle error in the above

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Reserves

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Answer: Error Fixed

Find the sid of sailors who have reserved exactly one boat

```
SELECT R3.sid FROM Reserves R3
MINUS
SELECT R1.sid
FROM Reserves R1, Boats B1, Reserves R2, Boats B2
WHERE R1.sid=R2.sid AND R1.bid=B1.bid
      AND R2.bid=B2.bid AND R1.bid <> R2.bid;
```

Answer: Error Fixed: Another Solution

Find the sid of sailors who have reserved exactly one boat

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid = R.sid AND B.bid = R.bid
GROUP BY S.sid
HAVING COUNT(*) = 1;
```

Find the age of the youngest sailor with age > 18, for each rating with at least 2 sailors (of any age)

```
SELECT  S.rating,  MIN (S.age) AS MINAGE
FROM  Sailors S
WHERE  S.age > 18
GROUP BY  S.rating
HAVING  1 < (SELECT  COUNT (*) FROM  Sailors S2
             WHERE  S2.rating=S.rating)
```

- Subquery in the HAVING clause
- Compare this with the query where we considered only ratings with 2 sailors over 18!

Find the age of the youngest sailor with age > 18, for each rating with at least 2 sailors (of any age)

```
SELECT  S.rating,  MIN (S.age) AS MINAGE
FROM  Sailors S
WHERE  S.age > 18
GROUP BY  S.rating
HAVING  1 < (SELECT  COUNT (*) FROM  Sailors S2
             WHERE  S2.rating=S.rating)
```

- Compare this with the query where we considered only ratings with 2 sailors over 18!

```
SELECT  S.rating, MIN (S.age)
FROM  Sailors S
WHERE  S.age >= 18
GROUP BY  S.rating
HAVING  COUNT (*) >= 2
```

Find ratings for which the average age is the minimum of the average age over all ratings*

Aggregate operations cannot be nested!
WRONG:

```
SELECT  S.rating
FROM  Sailors S
WHERE  AVG(S.age) =
  (SELECT  MIN (AVG (S2.age))
   FROM Sailors S2
   GROUP BY S2.rating)
```

Correct solution

```
SELECT  T.rating, T.avgage
FROM  (SELECT  S.rating, AVG (S.age) AS avgage
       FROM  Sailors S
       GROUP BY  S.rating) T
WHERE  T.avgage = (SELECT  MIN (T.avgage) FROM T);
```

Meets SQL/92 standard, but some products may not support this!

Solution Using Views

If previous solution does not work, you can define T as a view

```
CREATE VIEW AVG_AGE_BY_RATING AS
SELECT S.rating, AVG(S.age) AS avgage
FROM Sailors S
GROUP BY S.rating;


SELECT T.rating, T.avgage
FROM AVG_AGE_BY_RATING T
WHERE T.avgage= (SELECT MIN(A.avgage)
            FROM AVG_AGE_BY_RATING A);
```

```
DROP VIEW AVG_AGE_BY_RATING; -- for temporary view
```

Solution Using WITH

Can use “WITH” clause instead of defining a temporary VIEW

```
WITH AVG_AGE_BY_RATING AS
(SELECT S.rating, AVG(S.age) AS avgage
FROM Sailors S
GROUP BY S.rating)

SELECT T.rating, T.avgage
FROM AVG_AGE_BY_RATING T
WHERE T.avgage = (SELECT MIN (A.avgage)
                  FROM AVG_AGE_BY_RATING A);
```

CASE

```
CASE
  WHEN condition1 THEN value1
  WHEN condition2 THEN value2
  ELSE default_value
END
```

```
CASE expression
  WHEN option1 THEN value1
  WHEN option2 THEN value2
  ELSE default_value
END
```

What do the following queries return?

```
SELECT sid, score,
  CASE
    WHEN score >= 90 THEN 'A'
    WHEN score >= 80 THEN 'B'
    WHEN score >= 70 THEN 'C'
    ELSE 'D/F'
  END AS letter
FROM Grades;
```

```
SELECT
  COUNT(*) AS total,
  SUM(CASE
          WHEN color = 'red' THEN 1
          ELSE 0
        END) AS red_boats
FROM Boats;
```

Operator Precedence

- **Operator precedence (NOT > AND > OR)** is part of the SQL standard and is consistent across major DBs (Postgres, MySQL, SQL Server, Oracle, SQLite, Snowflake, ...), e.g..

```
WHERE C1 AND NOT C2 OR C3
```

Will be parsed as:

```
WHERE (C1 AND (NOT(C2)) OR C3
```

But as practice, try to always **use parentheses** for readability and to avoid mistakes.

Suggested Review

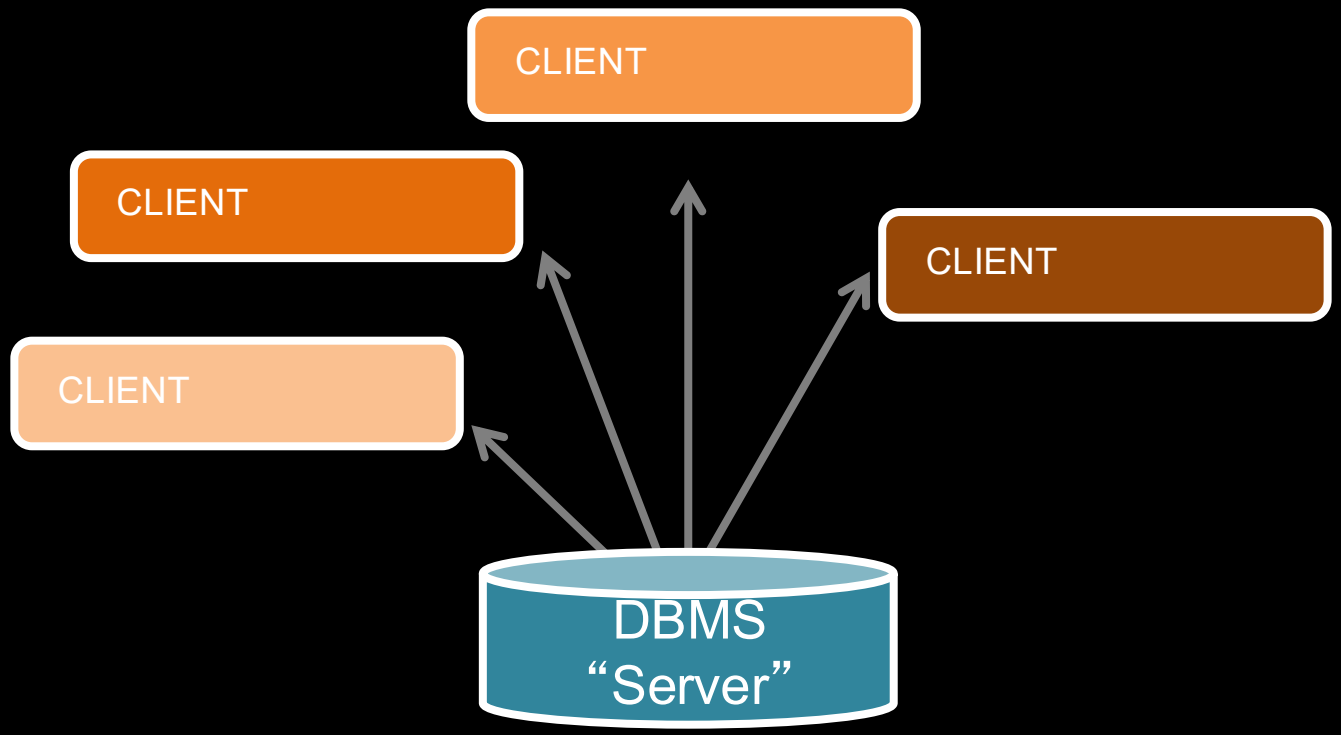
Exercises 5.1, 5.3, 5.7

Summary

- SQL implements relational algebra…

  Select, Project, Join, Set operators

- And so much more…
  - Correlated subqueries
  - Ordering of results
  - Aggregate queries (e.g., SUM, MAX, AVG)
  - Three-valued logic for NULL values
  - …

Next…

Database Application Programming